

CICIDS2017: การแบ่งข้อมูลแบบ "ตามลำดับเวลา" เรียงตามอะไร และสะท้อนเวลาจริงได้แค่ไหน

เอกสารนี้อธิบายการแบ่งข้อมูล chronological (ordered) split ของ CICIDS2017 ณ วันที่ 3 ตุลาคม 2569 ว่าเรียงตามอะไร และลำดับนั้นสะท้อนเวลาจริงได้แค่ไหน

ทุกตัวเลขอ่านมาจากไฟล์ใน results/split_audit/cicids2017/ ซึ่งสร้างด้วย scripts/audit_chronological_split_2017.py จากไฟล์ดิบใน data/raw/ และจากผลที่มีอยู่แล้วใน results/crossdataset/ ไม่ได้เทรนโมเดลหรือรันการทดลองใหม่

สรุปสั้น

- เรียงตามอะไร ตามตำแหน่งแถวของ flow ในไฟล์ CSV ดันฉบับแต่ละไฟล์ (`_row_index` นับจาก 0 ก่อนตัดแถวใดทิ้ง)
ไม่ใช่เวลาที่วัดได้ เพราะไฟล์ MachineLearningCVE ที่ใช้มี 79 คอลัมน์และไม่มี Timestamp การแบ่งทำแยกในทุกกลุ่ม (ไฟล์ที่จับข้อมูล, คลาส): แถวแรก 70% ของกลุ่มเป็น train แถวท้าย 30% เป็น test
- สะท้อนเวลาจริงได้แค่ไหน ได้ในระดับ "ช่วงการโจมตี" แต่ไม่ใช่นาฬิกา และมีจุดที่ขัดกับตารางเวลา
 - ที่ยืนยันได้ ลำดับแถวตรงกับตารางเวลาที่ CIC ประกาศครบทั้ง 7 คู่ (ถ้าไฟล์ถูกสลับแบบสุ่ม โอกาสผ่านทั้ง 7 คู่โดยบังเอิญคือ 1 ใน 1,440) และเมื่อตรวจเข้มข้นเป็น "มีแถวใดสลับหรือไม่" ใน 14 คู่ มี 13 คู่ที่ไม่มีแถวสลับเลย แม้ช่วงโจมตีต่างกันเพียง 4 ถึง 15 นาที และ `_row_index` ตรงกับตำแหน่งแถวในไฟล์ดิบครบทั้ง 2,497,980 แถวที่ clean แล้ว
 - ที่ยืนยันไม่ได้ ไม่มีเวลาจริงให้เทียบ จึงบอกไม่ได้ว่าห่างกันกี่นาที ลำดับเป็นลำดับที่ flow จบ ไม่ใช่ตอนเริ่ม (ตามพฤติกรรมของ CICFlowMeter ซึ่งไฟล์นี้พิสูจน์เองไม่ได้) และ flow ปกติไม่มีตารางเวลาให้ตรวจเลย
 - ที่ขัดกับตารางเวลา 1 ใน 14 คู่: ใน Wednesday แถว DoS GoldenEye 4,939 จาก 10,293 แถว (48.0%) อยู่หลังแถว Heartbleed แถวแรก ทั้งที่ตารางเวลาให้ GoldenEye (11:10 ถึง 11:23) มาก่อน Heartbleed (15:12 ถึง 15:32) ราว 4 ชั่วโมง การตรวจที่รันก่อนเทรนมองไม่เห็น เพราะตรวจเพียง 1 คู่และดูแค่คำมีชยฐาน
- ผลต่อ test set ใน CICIDS2017 ลำดับแถวตรงกับลำดับของ sub-attack ภายในคลาส "30% ท้าย" ของคลาสจึงมักเป็น sub-attack ตัวท้ายที่ train เห็นน้อยหรือไม่เห็นเลย: test ของ Brute Force เป็น SSH-Patator ล้วน, test ของ Web Attack เป็น XSS และ SQL Injection ล้วน, และ DoS GoldenEye 10,282 แถว (17.7% ของ test ของ DoS) ไม่อยู่ใน train เลย ผลคือเพดาน (ceiling) ของ 2017 แบบ chronological ในงาน cross-dataset ถูกกดโดย Brute Force ที่ได้ $F1 \approx 0$ ในทุกโมเดล ดูหัวข้อ 5
- ในรายงาน ควรเรียกว่า "แบ่งตามลำดับการบันทึกในไฟล์ (capture order) ภายในแต่ละกลุ่ม (ไฟล์, คลาส)" ไม่ควรเรียกว่า แบ่งตาม timestamp หรือ "เทรนบนอดีต ทดสอบบนอนาคต" โดยไม่มีเงื่อนไข ดูหัวข้อ 6

1. เรียงตามอะไร

ข้อ	รายละเอียด
ตัวบอกลำดับ	<code>_row_index</code> คือตำแหน่งแถวข้อมูลในไฟล์ CSV นับจาก 0 (ไม่นับ header) กำหนดใน <code>train_clean_one_frame</code> ทันทีหลังอ่านไฟล์ ก่อนตัดแถวใดทิ้ง จึงเป็นตำแหน่งดิบแม้แถวรอบข้างถูกตัดไปแล้ว
ใช้ภายในไฟล์เท่านั้น	นับใหม่จาก 0 ในทุกไฟล์ ลำดับข้ามไฟล์ไม่ได้ใช้การแบ่งทำภายในกลุ่ม (ไฟล์, คลาส)

ข้อ	รายละเอียด
ทำไมไม่ใช้เวลา	ไฟล์ MachineLearningCVE มี 79 คอลัมน์ ไม่มี Timestamp, Flow ID, Source/Destination IP (ตรวจ header ของไฟล์ดิบแล้ว)
_row_index ตรงแถวดิบจริงไหม	เทียบทุกแถวที่ clean แล้ว 2,497,980 แถวกับแถวดิบที่ตำแหน่งนั้นด้วย 3 คอลัมน์ (Destination Port, Flow Duration, Total Fwd Packets): ไม่ตรง 0 แถว, อยู่นอกช่วง 0, index ซ้ำ 0 (ไฟล์ดิบรวม 2,830,743 แถว)
การแบ่ง	เรียงแถวของกลุ่มตาม _row_index แล้วตัด 70/30 ไม่ใช่ตัวสุ่ม ผลซ้ำได้ทุกครั้ง และสร้างตัวเลขตรงกับ manifest (train 1,748,579, test 749,401)

ใน protocol ของ cross-dataset ใช้ตัวบอกลำดับเดียวกันสำหรับ 2017 แต่ตัดเป็น 60/10/30 (train/calibration/test) ส่วน 2018 มี Timestamp จริงจึงใช้ Timestamp

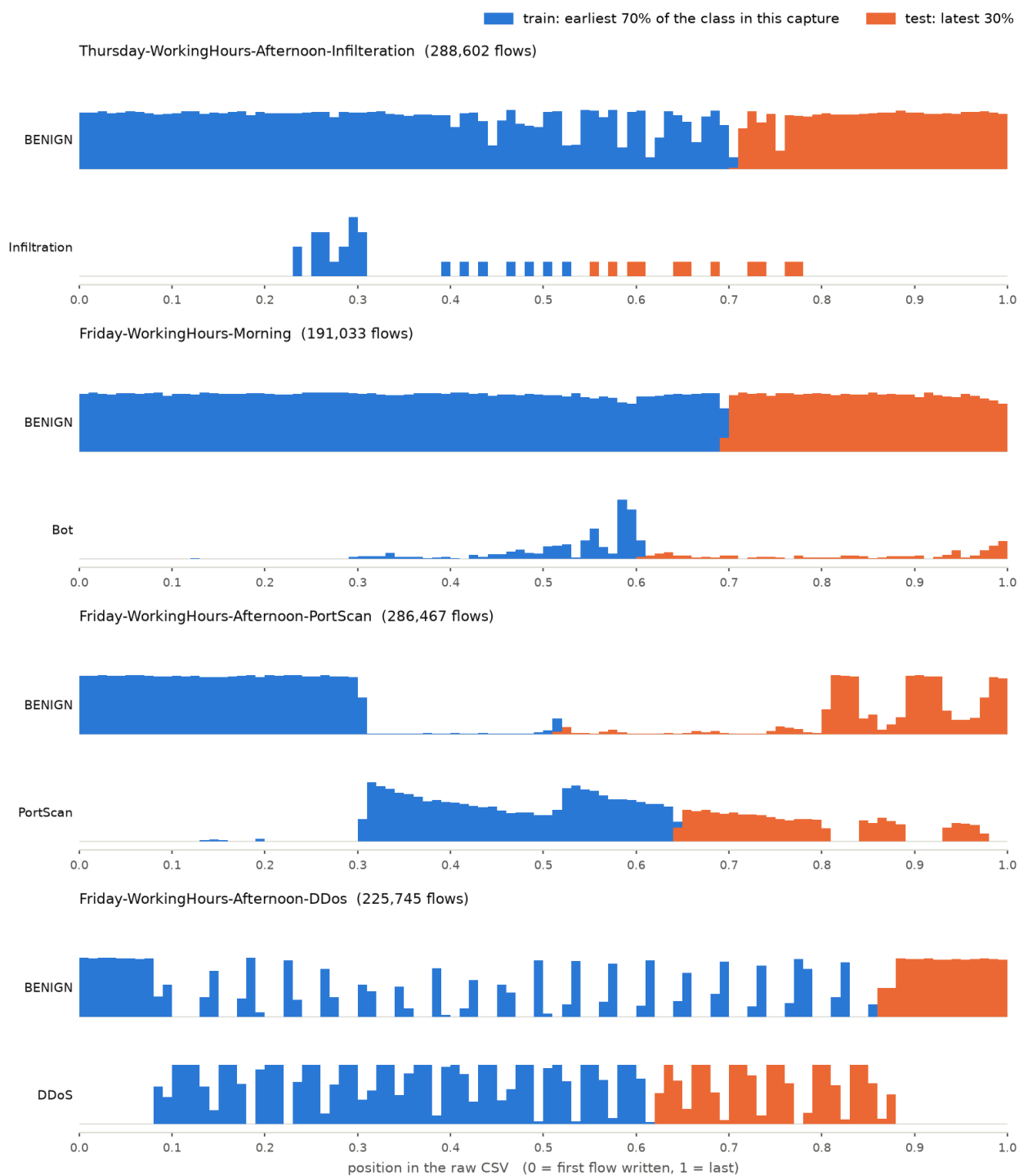
2. การแบ่งใช้ลำดับนี้อย่างไร

ทำแยกในทุก (ไฟล์, คลาส) เพราะแต่ละคลาสโจมตีอยู่ในไฟล์เดียว ถ้าตัดทั้งไฟล์ด้วยเวลาเดียว บางคลาสจะไม่มีข้อมูล train เลย (เช่น Heartbleed 11 แถวอยู่ที่ตำแหน่ง 0.862 ของ Wednesday)

ตัวอย่าง Wednesday: DoS มี 193,730 แถวหลัง clean เรียงตามตำแหน่งแล้ว 135,611 แถวแรกเป็น train 58,119 แถวท้ายเป็น test แถว test แถวแรกของ DoS อยู่ที่ตำแหน่ง 0.358 ของไฟล์ ส่วน BENIGN ของไฟล์เดียวกัน ถูกตัดที่ตำแหน่ง 0.787 จุดตัดของแต่ละคลาสอยู่ที่ตำแหน่งต่างกัน ไม่ใช่เส้นเวลาเดียว



รูปที่ 1 ตำแหน่งแถวของแต่ละคลาสในไฟล์ดิบ สีฟ้าคือ train สีส้มคือ test (ไฟล์ที่มีหลาย sub-attack)



รูปที่ 2 เช่นเดียวกับรูปที่ 1 สำหรับไฟล์ที่เหลือ

แกนนอนคือลำดับแถว ไม่ใช่เวลา ช่วงที่มี flow โจมตีหนาแน่น flow ปกติจะดูบางลง (เช่น BENIGN ของ Wednesday ช่วงตำแหน่ง 0.1 ถึง 0.45) เพราะแกนนี้นับจำนวน flow ไม่ใช่เวลาที่

ตำแหน่งที่ test เริ่มของกลุ่มโจมตีทุกกลุ่ม:

ไฟล์	คลาส	แถว (หลัง clean)	train / test	test เริ่มที่ตำแหน่ง	ช่วงเวลาที CIC ประกาศ
Tuesday	Brute Force	9,150	6,405 / 2,745	0.393	FTP-Patator 09:20-10:20; SSH-Patator 14:00-15:00
Wednesday	DoS	193,730	135,611 / 58,119	0.358	DoS slowloris 09:47-10:10; DoS Slowhttptest 10:14-10:35; DoS Hulk 10:43-11:00; DoS GoldenEye 11:10-11:23
Wednesday	Heartbleed	11	7 / 4	0.863	Heartbleed 15:12-15:32
Thursday เช้า	Web Attack	2,143	1,500 / 643	0.426	Web Attack Brute Force 09:20-10:00; Web Attack XSS 10:15-10:35; Web Attack Sql Injection 10:40-10:42
Thursday บ่าย	Infiltration	36	25 / 11	0.554	Infiltration 14:19-15:45
Friday เช้า	Bot	1,948	1,363 / 585	0.603	Bot 10:02-11:02
Friday บ่าย PortScan	PortScan	90,694	63,485 / 27,209	0.646	PortScan 13:55-15:29
Friday บ่าย DDoS	DDoS	128,014	89,609 / 38,405	0.625	DDoS 15:56-16:16

ไม่มีเส้นเวลาเดียวที่แบ่ง train จาก test ทั้งไฟล์ สัดส่วนของแถว train ที่อยู่หลังแถว test แถวแรกของไฟล์เดียวกัน:

ไฟล์	แถว train	แถว train ที่อยู่หลังแถว test แถวแรกของไฟล์	สัดส่วน
Wednesday	410,613	207,225	50.5%
Tuesday	284,526	120,009	42.2%
Thursday เช้า	108,119	41,834	38.7%
Thursday บ่าย	169,707	31,594	18.6%
Friday บ่าย PortScan	148,577	26,124	17.6%
Friday เช้า	126,058	16,837	13.4%
Friday บ่าย DDoS	156,156	16,314	10.4%
Monday	344,823	0	0.0%

ใน Wednesday แถว train 50.5% อยู่หลังแถว test แถวแรกของไฟล์ (ตำแหน่ง 0.358) นั่นคือ ขั้รับประกัน "ไม่มี test flow ไตอยู่ก่อน train flow" เป็นจริงเฉพาะภายในคลาสเดียวกันของไฟล์เดียวกัน

3. หลักฐานว่าลำดับแถวตามเวลา

3.1 การตรวจที่มีอยู่กับการตรวจที่เอกสารอ้าง

การตรวจ	ใช้ label ไหน	จำนวนคู่	รันเมื่อไร	ผล
validate_capture_chronology	label 9 คลาสที่ยับ sub-type แล้ว	1 (DoS ก่อน Heartbleed)	ก่อนเทรนทุกครั้ง และใน --stage audit --verify-data	ผ่าน
validate_raw_label_chronology	sub-label ดิบ (FTP-Patator, slowloris ฯลฯ)	7	ไม่ถูกเรียกที่ใดในไปป์ไลน์ รันด้วยมือในครั้งนี้	ผ่านทั้ง 7 คู่
ตรวจแบบ "มีแถวใดสลับหรือไม่" (สคริปต์ audit นี้)	sub-label ดิบ	14 (ทุกคู่ตามตารางเวลา)	รันในครั้งนี้	13 คู่ไม่มีแถวสลับ, 1 คู่ไม่ผ่าน

เอกสารส่งมอบของ 2017 (README_HANDOFF.md, docs/experiments/ids2017/README.md) ระบุว่า "ตรวจก่อนเทรนทุกครั้ง" ซึ่งจริงสำหรับการตรวจแรก (1 คู่) เท่านั้น การตรวจ 7 คู่ที่เอกสารอ้างว่า "พิสูจน์" ลำดับมีในโค้ดแต่ไม่ได้ถูกเรียกใช้ ผลที่ได้ครั้งนี้ผ่านทั้ง 7 คู่

สองการตรวจแรกเทียบค่ามัธยฐานตำแหน่งของแต่ละ sub-label กับลำดับที่ประกาศ ซึ่งผ่านได้แม้ sub-label นั้นมีแถวจำนวนมากอยู่ผิดที่

3.2 คู่ที่ติดกันตามตารางเวลา

ไฟล์	คู่ (ก่อน → หลัง)	ห่างกันตามตารางเวลา	แถว (ก่อน / หลัง)	คู่แถวที่ลำดับสลับ	แถวของตัวก่อนที่อยู่หลังตัวหลังเริ่ม
Thursday เช้า	Web Attack Brute Force → Web Attack XSS	15 นาที	1,507 / 652	0.0%	0 (0.0%)
Thursday เช้า	Web Attack XSS → Web Attack Sql Injection	5 นาที	652 / 21	0.0%	0 (0.0%)
Tuesday	FTP-Patator → SSH-Patator	3.7 ชั่วโมง	7,938 / 5,897	0.0%	0 (0.0%)
Wednesday	DoS slowloris → DoS Slowhttptest	4 นาที	5,796 / 5,499	0.0%	0 (0.0%)
Wednesday	DoS Slowhttptest → DoS Hulk	8 นาที	5,499 / 231,073	0.0%	0 (0.0%)
Wednesday	DoS Hulk → DoS GoldenEye	10 นาที	231,073 / 10,293	0.0%	0 (0.0%)
Wednesday	DoS GoldenEye → Heartbleed	3.8 ชั่วโมง	10,293 / 11	47.8%	4,939 (48.0%)

คู่ที่ไม่ติดกันอีก 7 คู่ไม่มีแถวสลับเลยเช่นกัน ช่วงโจมตีที่ห่างกันเพียง 4 ถึง 15 นาทียังแยกกันสะอาด (ไม่มีแถวของตัวก่อนอยู่หลังแถวแรกของตัวหลัง) นั่นคือความละเอียดของลำดับแถวระดับ "ก่อนการโจมตี" ที่ตรวจได้ ความละเอียดภายในก่อนไม่มีอะไรตรวจ

3.3 จุดที่ขัดกับตารางเวลา: DoS GoldenEye

ตารางเวลา CIC: GoldenEye 11:10 ถึง 11:23, Heartbleed 15:12 ถึง 15:32 แต่ในไฟล์ดิบ GoldenEye ทั้ง 10,293 แถวแตกเป็น 3 ก้อน (ตัดที่ช่องว่างกว้างกว่า 5% ของไฟล์):

ก้อน	แถวในไฟล์ดิบ	ตำแหน่งในไฟล์	พอร์ตปลายทางหลัก	TCP window เริ่มต้น (fwd / bwd)	ระยะ flow (วินาที) มัธยฐาน / p95 / สูงสุด
1	1,118	0.478 ถึง 0.480	80 (100%)	29,200 / 235	17.5 / 24.5 / 73.6
2	6	0.596 ถึง 0.596	80 (100%)	29,200 / 235	11.1 / 11.3 / 11.4
3	9,169	0.744 ถึง 1.000	80 (100%)	29,200 / 235	11.5 / 100.9 / 119.6

- ก้อนที่ 1 (1,118 แถว) ต่อท้าย Hulk ทันที สอดคล้องกับตารางเวลา
- ก้อนที่ 3 (9,169 แถว, 89.1% ของ GoldenEye) กระจายสม่ำเสมอจากตำแหน่ง 0.744 จนถึงแถวสุดท้ายของไฟล์ รวมถึงช่วงหลัง Heartbleed (4,939 แถวอยู่หลังแถว Heartbleed แถวแรก และทั้ง 11 แถวของ Heartbleed อยู่ก่อนแถว GoldenEye แถวสุดท้าย)
- ค่าอธิบายเดิมในเอกสารส่งมอบ (GoldenEye เปิด connection ค้างไว้ record จึงตามหลังช่วงโจมตี) อธิบายส่วนนี้ไม่ได้ flow ของ 4,939 แถวที่อยู่หลัง Heartbleed มีระยะมัธยฐาน 11.5 วินาที, p95 100.5 วินาที, สูงสุด 107.8 วินาที ขณะที่ flow ที่ยาวที่สุดในทุกไฟล์คือ 120 วินาที ถ้าลำดับแถวคือลำดับที่ flow จม flow ที่ค้างอย่างมากสองนาทีก่อนทำให้แถวไปอยู่หลังช่วงโจมตีหลายชั่วโมง
- ลายเซ็นของก้อนที่ 3 เหมือนก้อนที่ 1: ปลายทางพอร์ต 80 ทุกแถวและ TCP window เริ่มต้นชุดเดียวกัน จึงไม่น่าเป็นแถวสุ่มที่ติด label ผิดแบบกระจาย แต่สอดคล้องกับ traffic ชนิดเดียวกัน (client และ service แบบเดียวกัน)

สมมติฐานที่เป็นไปได้ (ข้อมูลที่มีแยกไม่ได้ว่าข้อใด): (ก) label ผูกกับคู่ IP ไม่ใช่ช่วงเวลา แถวหลังช่วงโจมตีจึงยังได้ label เดิม (ข) การโจมตียังดำเนินต่อนอกตารางที่ประกาศ (ค) ลำดับแถวช่วงท้ายไฟล์ไม่ใช่ลำดับเวลา ทั้งสามข้อให้ข้อมูลหน้าตาเหมือนกันในไฟล์ที่ไม่มี Timestamp

ลักษณะเดียวกันพบที่ Tuesday SSH-Patator ซึ่งเป็นการโจมตีสุดท้ายของไฟล์ ตารางเวลาจึงไม่มีคู่ให้เทียบ แต่รูปร่างเหมือนกัน (ช่วง 14:00 ถึง 15:00 ที่ประกาศ):

ก้อน	แถวในไฟล์ดิบ	ตำแหน่งในไฟล์	พอร์ตปลายทางหลัก	TCP window เริ่มต้น (fwd / bwd)	ระยะ flow (วินาที) มัธยฐาน / p95 / สูงสุด
1	2,973	0.363 ถึง 0.469	22 (100%)	29,200 / 247	1.9 / 13.4 / 119.8
2	1	0.528 ถึง 0.528	22 (100%)	259 / 247	0.0 / 0.0 / 0.0
3	2,923	0.622 ถึง 1.000	22 (100%)	29,200 / 247	10.3 / 13.6 / 19.6

ก้อนที่ 3 ของ SSH-Patator (2,923 แถว) กระจายสม่ำเสมอตั้งแต่ตำแหน่ง 0.622 ถึงท้ายไฟล์ ใช้พอร์ต 22 และ TCP window เหมือนก้อนที่ 1 แต่ flow ยาวกว่า (มัธยฐาน 10.3 วินาที เทียบ 1.9)

4. ข้อจำกัด: ไม่ใช่นาฬิกา

1. ไม่มีเวลาจริง ลำดับบอกว่าจะไรมาก่อนอะไร ไม่บอกว่าห่างกันเท่าไร ตัวเลขเวลาเดียวที่มีคือช่วงที่ CIC ประกาศ ซึ่งเป็นช่วงโดยประมาณ
2. ลำดับเป็นลำดับที่ flow จม ไม่ใช่ตอนเริ่ม ตามพฤติกรรมของ CICFlowMeter ที่เขียน record เมื่อ flow สิ้นสุด ไฟล์ชุดนี้ไม่มีคอลัมน์ที่ใช้ยืนยันข้อนี้เอง
3. ตรวจได้เฉพาะระดับก่อนการโจมตี ภายในก้อน และ flow ปกติ (ซึ่งไม่มีตารางเวลา) ไม่มีอะไรตรวจ
4. แกนแถวไม่เป็นเส้นตรงกับเวลา ช่วงที่ flow โจมตีหนาแน่นกินพื้นที่แถวมากกว่าเวลาที่ใช้ (รูปที่ 1 และ 2)

5. แล้วย้อมตีส่วนหนึ่งอยู่นอกช่วงที่ประกาศ และการตรวจที่ใช้มีฐานมองไม่เห็น (หัวข้อ 3.3)
6. ลำดับของแต่ละไฟล์ใช้ได้ภายในไฟล์เท่านั้น ข้ามไฟล์ไม่ได้ใช้
7. ไม่ได้ตรวจกับไฟล์ที่มี Timestamp CIC แจกไฟล์ CICIDS2017 อีกชุดที่มีคอลัมน์เวลาต่อ flow ถ้าจับคู่แถวกับชุดนั้นได้ จะตรวจลำดับระดับวินาที และดูได้ว่าแถวท้ายไฟล์ของ GoldenEye และ SSH-Patator เกิดเมื่อไร แต่ยังไม่ได้หาไฟล์ชุดนั้นหรือตรวจในรอบนี้

5. ผลต่อ test set: ลำดับเวลา = ลำดับของ sub-attack

5.1 แบ่ง 70/30 ของบันทึก cecids2017_temporal_v1

คลาส	sub-attack (ตามลำดับในไฟล์)	train	test
Brute Force	FTP-Patator	5,931	0
Brute Force	SSH-Patator	474	2,745
DoS	DoS slowloris	5,374	0
DoS	DoS Slowhttpstest	5,228	0
DoS	DoS Hulk	125,009	47,837
DoS	DoS GoldenEye	0	10,282
Web Attack	Web Attack Brute Force	1,470	0
Web Attack	Web Attack XSS	30	622
Web Attack	Web Attack Sql Injection	0	21

(ตัวเลขหนาคือ 0 แถว)

- Brute Force: test (2,745 แถว) เป็น SSH-Patator ทั้งหมด train มี FTP-Patator 5,931 แถว และ SSH-Patator เพียง 474 แถว ในจำนวน test 1,211 แถวมาจากก่อนที่ 1 และ 1,534 แถวมาจากก่อนที่ 3 ซึ่งอยู่หลังก่อนหลักในลำดับแถว
- Web Attack: test 643 แถวเป็น XSS 622 และ SQL Injection 21 ทั้งหมด train มี XSS เพียง 30 แถวและไม่มี SQL Injection เลย
- DoS: GoldenEye 10,282 แถว (17.7% ของ test ของ DoS) อยู่ใน test ทั้งหมดและไม่อยู่ใน train เลย ในจำนวนนี้ 9,160 แถวมาจากก่อนที่ 3 ซึ่งอยู่หลัง Heartbleed ในลำดับแถว
- Bot: test 585 แถว อยู่ที่ตำแหน่ง 0.603 ถึง 1.000 ซึ่งเป็นส่วนหางหลังก่อนหลักของ Bot (รูปที่ 2) recall ของ Bot ที่ติดอยู่ที่ 0.150 ถึง 0.161 ในทุกโมเดล (docs/experiments/ids2017/README.md สรุปว่าเพดานอยู่ที่ข้อมูล) สอดคล้องกับลักษณะนี้ แต่ยังไม่ได้ตรวจว่าเป็นสาเหตุ

5.2 แบ่ง 60/10/30 ของ cross-dataset (ตัวอย่าง 2017 จำนวน 300,000 แถว)

คลาส	sub-attack	train	calibration (ไม่ได้ใช้)	test
Brute Force	FTP-Patator	5,490	441	0
Brute Force	SSH-Patator	0	474	2,745
Web Attack	Web Attack Brute Force	1,286	184	0
Web Attack	Web Attack XSS	0	30	622

คลาส	sub-attack	train	calibration (ไม่ได้ใช้)	test
Web Attack	Web Attack Sql Injection	0	0	21
DoS	DoS slowloris	682	0	0
DoS	DoS Slowhttptest	597	0	0
DoS	DoS Hulk	12,643	2,320	5,735
DoS	DoS GoldenEye	0	0	1,226

Brute Force: train เป็น FTP-Patator ล้วน (5,490 แถว) SSH-Patator 474 แถวแรกตกอยู่ในช่อง calibration ที่ยังไม่ได้ใช้ และ test (2,745 แถว) เป็น SSH-Patator ทั้งหมด ถ้าแบ่งแบบสุ่ม ทั้ง train และ test มีทั้งสองชนิด (test: FTP-Patator 1,766 และ SSH-Patator 979)

ผลที่ตามมาของ Brute Force ในพีดานของ 2017 (protocol_v2 เฉลี่ย 5 seeds):

โมเดล	Brute Force F1 แบบ chronological (60/10/30)	Brute Force F1 แบบสุ่ม	Brute Force recall ในบันเดิล 70/30 (train มี SSH-Patator)
catboost	0.0013	0.9994	0.9985
lightgbm	0.0000	0.9995	0.9960
logistic_regression	0.0043	0.9678	0.9581
mlp	0.0021	0.9806	0.9621
random_forest	0.0000	0.9992	0.9767
stacking	0.0000	0.9995	0.9971
xgboost	0.0000	0.9997	0.9989

โมเดลที่เทรนด้วย FTP-Patator ล้วนตรวจ SSH-Patator ไม่ได้เลย (F1 0.0000 ถึง 0.0043) ขณะที่ในบันเดิล 70/30 ซึ่ง train มี SSH-Patator 474 แถว recall อยู่ที่ 0.9581 ถึง 0.9989 สอดคล้องกับว่าเลข F1 ≈ 0 ของ Brute Force ในพีดานแบบ chronological มาจากการที่ train ไม่มี SSH-Patator เพราะช่อง calibration กันออกไป ไม่ใช่จากความยากของคลาส (หลักฐานทางอ้อม: บันเดิล 70/30 ต่างกันหลายอย่าง ทั้งขนาด train และตัวชี้วัด ยังไม่ได้แยกทดสอบ)

5.3 ส่วนต่างระหว่างพีดานแบบสุ่มกับ chronological ของ 2017 มาจากคลาสใด

รายงานรอบ 4 และ 5 อธิบายว่าการแบ่งแบบสุ่มทำให้พีดานของ 2017 เฟือ (0.21 ถึง 0.29 ใน protocol_v2) macro-F1 เป็นค่าเฉลี่ยของ F1 รายคลาส ส่วนต่างจึงแยกเป็นรายคลาสได้ตรงตัว:

โมเดล	พีดาน 2017 เฟือขึ้น (สุ่ม – chronological)	จาก Brute Force	จาก Bot	จากอีก 5 คลาสรวมกัน	Brute Force + Bot เป็นสัดส่วน
lightgbm	0.2450	0.1428	0.0988	+0.0034	98.6%
catboost	0.2534	0.1426	0.0993	+0.0115	95.5%
stacking	0.2526	0.1428	0.0972	+0.0126	95.0%
xgboost	0.2547	0.1428	0.0989	+0.0130	94.9%
random_forest	0.2915	0.1427	0.0972	+0.0516	82.3%

โมเดล	เพดาน 2017 เฟอซัน (สุม – chronological)	จาก Brute Force	จาก Bot	จากอีก 5 คลาสรวมกัน	Brute Force + Bot เป็นสัดส่วน
mlp	0.2884	0.1398	0.0837	+0.0649	77.5%
logistic_regression	0.2057	0.1376	0.0052	+0.0629	69.4%

Brute Force กับ Bot สองคลาสอธิบาย 69.4% ถึง 98.6% ของส่วนต่าง

ส่วนต่างระหว่างสองโมเดลจึงส่วนใหญ่เป็นผลที่ test แบบ chronological ของสองคลาสนี้เป็น sub-attack หรือช่วงท้ายที่ train ไม่เห็น ไม่ใช่ผลที่กระจายทุกคลาส การแบ่งแบบสุมให้ test อยู่ในการกระจายเดียวกับ train (เพดานสูง และอาจรวมแถวที่ใกล้เข้ากัน) ส่วน chronological ให้ test เป็นช่วงที่ต่างออกไป (เพดานต่ำ) ข้อมูลที่มีแยกไม่ได้ว่าส่วนต่างของสองคลาสนี้เป็นผลของการรั่ว หรือผลของช่วงที่ต่างออกไป อย่างละเท่าไร

เพดานแบบ chronological จึงไม่ใช่ "เพดานที่ไม่รั่ว" เพียงอย่างเดียว แต่รวมผลของการที่ train ขาด sub-attack บางตัวด้วย

ผลต่อ Recovery ทิศ 2018 ไป 2017 ของ lightgbm (เลขคณิตเพื่อดูขนาด ไม่ใช่ผลการทดลอง): ถ้า Brute Force ได้ 0.9995 เท่าแบบสุม เพดานจะเป็นราว 0.8753 แทน 0.7325 และ Recovery จะลดจาก 52.8% เป็นราว 42.7% ส่วนคะแนนข้ามชุด (0.4480 ± 0.0079) ไม่เปลี่ยน ดูรายงาน docs/experiments/crossdataset/round5_reliability_audit/report/macro_f1_with_recovery_20261003.md

6. ควรเขียนในรายงานอย่างไร

ใช้ได้

> แบ่งตามลำดับการบันทึกในไฟล์ (capture order) ภายในแต่ละกลุ่ม (ไฟล์ที่จับข้อมูล, คลาส): 70% แรกของคลาสในไฟล์นั้นเป็น train > 30% ท้ายเป็น test ตัวออกลำดับคือตำแหน่งแถวในไฟล์ CSV ต้นฉบับ เพราะไฟล์ MachineLearningCVE ไม่มี Timestamp ลำดับนี้ตรงกับตารางเวลาที่ CIC > ประกาศในระดับช่วงการโจมตี แต่ไม่ใช่เวลาที่วัดได้

ไม่ควรเขียน

ข้อความ	ทำไม
"แบ่งตาม timestamp"	ไม่มี Timestamp ในไฟล์ที่ใช้
"เทรนบนอดีต ทดสอบบนอนาคต" โดยไม่มีเงื่อนไข	ในไฟล์ที่มีการโจมตี แถว train 10.4% ถึง 50.5% อยู่หลังแถว test แถวแรกของไฟล์ (หัวข้อ 2) จริงเฉพาะภายในคลาสเดียวกัน
"ไม่มี test flow ใดอยู่ก่อน train flow"	จริงเฉพาะคลาสเดียวกันในไฟล์เดียวกัน
"ลำดับถูกพิสูจน์ว่าตรงกับเวลา"	ตรวจได้ระดับก่อนการโจมตี และมี 1 คู่ที่ขัด (หัวข้อ 3)
"chronological ceiling คือเพดานที่ไม่รั่ว"	รวมผลของ sub-attack ที่ train ไม่เห็นด้วย (หัวข้อ 5)

เมื่อรายงานผลรายคลาสของ DoS, Brute Force, Web Attack ควรระบุว่า test เป็น sub-attack ที่ train เห็นน้อยหรือไม่เห็น (หัวข้อ 5.1)

7. ไฟล์และวิธีสร้างซ้ำ

ไฟล์	เนื้อหา
results/split_audit/cicids2017/row_order_audit.json	ผลทั้งหมด: ตำแหน่งของทุก sub-label, ก่อน, การตรวจลำดับทุกคู่, การเทียบ _row_index, การสร้างซ้ำ manifest

ไฟล์	เนื้อหา
results/split_audit/cicids2017/group_boundaries.csv	ต่อ (ไฟล์, คลาส): จำนวน train/test และตำแหน่งที่ test เริ่ม
results/split_audit/cicids2017/subtype_composition.csv	ต่อ (ไฟล์, คลาส, sub-label, ก้อน): จำนวน train/test
results/split_audit/cicids2017/crossdataset_partition_2017.csv	องค์ประกอบ sub-attack ของการแบ่ง 60/10/30 และการแบ่งแบบสุ่ม
results/split_audit/cicids2017/inflation_by_class.csv	ส่วนต่างเพดานรายคลาส (v1 และ v2)
docs/experiments/ids2017/data/split_audit/	สำเนาของทั้งหมดข้างบน
docs/experiments/ids2017/report/figures/	รูปที่ 1 และ 2

```
python scripts/audit_chronological_split_2017.py --figure-dir docs/experiments/ids2017/report/figures
```

ใช้เวลาราว 1 นาที ต้องมี data/raw/ และ data/processed/cicids2017_clean.parquet